Phronesis

Adaptive AI practice to rebuild code comprehension

Name: Emmanuel Phillips
Student ID: 230355585
Module: IOT6000U Digital Innovation and Product Development
Assessment: 001 Presentation

Area of Investigation: AI is outpacing understanding

Vibe coding: using artificial intelligence, prompted by natural language, to assist with writing computer code (Collins Dictionary, 2025)

84%
of developers use or plan to use AI tools, 51% of professionals daily

(Stack Overflow, 2025)

50% vs 67%
comprehension quiz score, AI-assisted vs hand-coding developers, biggest gap on debugging

(Shen and Tamkin, 2026)

17% lower
grades once AI access was removed, for students with unrestricted GPT-4

(Bastani *et al.*, 2025)

How AI is used matters: designs that keep learners engaged largely avoided the harm (Bastani *et al.*, 2025; Shen and Tamkin, 2026)

Gap in current solutions

What exists	What it offers	What it misses
Interview platforms (e.g. LeetCode)	Huge fixed problem bank	Built around passing interviews
AI add-ons (e.g. LCPath, LeetCode Coach)	Weakness analysis and hints (LeetCode Coach, 2026)	Aimed at interview candidates, still centred on solving problems
Course platforms (e.g. Codecademy)	AI tutoring	Set curriculum
Knowledge tracing research	Models learner progress over time	Mostly predicts correctness from quiz-style data (Dai *et al.*, 2021)

The gap: adaptive practice aimed at comprehension (reading, tracing and debugging code). Tracing and explaining skills are linked to code writing ability (Lopez *et al.*, 2008)

Proposed AI solution

A daily chain of tasks on one codebase: fix, refactor, reason, extend

1. Learner model

Bayesian knowledge tracing tracks mastery per concept and skill, and picks what to practise next

(Corbett and Anderson, 1994)

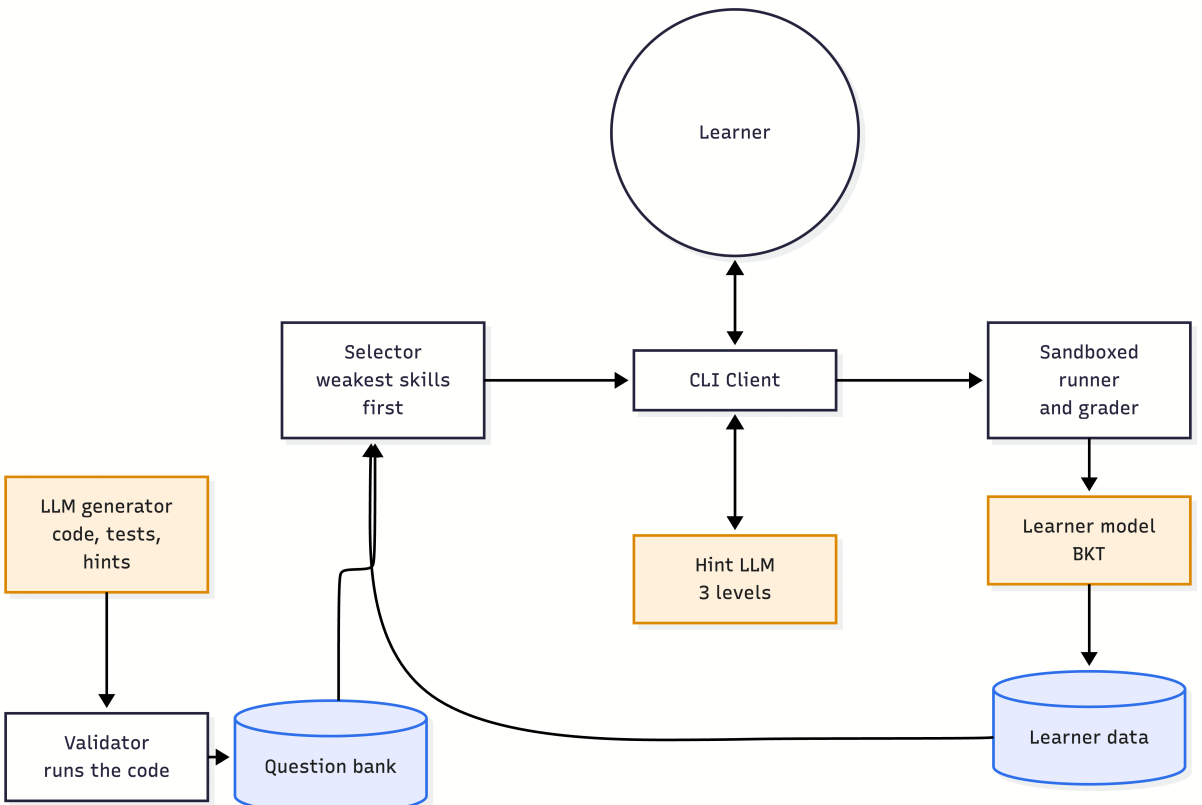
2. LLM content

Generates code, tests and bugs. Every task is run and validated before a learner sees it

3. Guided hints

Three levels that nudge without giving the answer, to keep you thinking

Prototype and feasibility



Proof of concept: validate, learn, adapt

Alternative solution comparison

A simpler approach: a curated question bank with rules-based personalisation

Feature	Phronesis	Rules-based alternative
Learner model	Bayesian knowledge tracing	Fixed rules based on errors
Question source	LLM-generated + curated	Curated question bank
Personalisation	Updates mastery after each attempt	Selects tasks using predefined rules
Advantages	Flexible adaptation and greater variety	Simpler, cheaper and easier to validate
Limitations	Higher cost and validation complexity	Less flexible; manual question authoring

Competitive advantage

Combines adaptive mastery tracking, validated code generation and graduated hints in one feedback loop. Educational impact still requires user testing.

Obstacles: external (PESTEL)

PESTEL examines external factors beyond the project's direct control (OpenStax, 2019)

Factor	Risk to Phronesis
Political	Changes in AI and education policy may affect deployment
Economic	LLM inference costs may limit affordable daily use
Social	Learners may distrust AI feedback or over-rely on hints
Technological	Incorrect generated code or tests could teach misconceptions
Environmental	Repeated model inference consumes computing resources
Legal	UK GDPR and copyright obligations affect data and content

Mitigation strategy

Minimise personal data, validate generated tasks, monitor API costs and review legal obligations before deployment.

Obstacles: internal

Initial qualitative risk assessment; prioritise risks by likelihood and impact (Tabassi, 2023)

Risk	Likelihood	Impact	Mitigation
Incorrect generated tasks	High	High	Execute code, validate tests and review content
Cold-start learner model	High	Medium	Use a short diagnostic assessment and conservative priors
Poor learner retention	Medium	High	Short sessions, progress tracking and user testing
API costs and capacity	Medium	High	Usage limits, caching and appropriate model selection
Limited development time	High	Medium	Build the core learning loop first; defer advanced features

Priority

Validate question quality first: unreliable tasks undermine both learner trust and the accuracy of the learner model.

Business case: initiative and value

Initiative

Start as an internal learning pilot for novice programmers. Validate educational impact before wider investment.

Learner value

Build code comprehension, debugging and reasoning skills through targeted daily practice.

Organisational value

Track skill development and target learning support at individual weaknesses.

Competitive advantage

Combines mastery tracking, validated code tasks and graduated hints in one learning loop.

Path to scale

Internal pilot, then interview and CTF preparation, then corporate learning and development

Business case: resources

Short term: pilot

People: solo developer, volunteer learners and an academic reviewer
Technology: Python, existing LLM API, SQLite, automated tests and a basic UI
Costs: development time, API usage and hosting, kept down with usage limits

Long term: deployment

People: software engineer, AI/ML expertise and learning-design support
Technology: hosted backend, database, monitoring, authentication and a secure execution sandbox
Costs: hosting, model inference, maintenance, security and ongoing content validation

Build versus buy: build the adaptive learning loop; use existing models and infrastructure rather than training a foundation model from scratch.

Conclusion

Better code generation doesn’t guarantee better understanding.

Phronesis uses adaptive practice, validated AI-generated tasks and graduated hints to help learners understand code, not just produce it.

The next step

Evaluate learning outcomes and cost in a small pilot before scaling.

References

Bastani, H. *et al.* (2025) "Generative AI without guardrails can harm learning: Evidence from high school mathematics," *Proceedings of the National Academy of Sciences*, 122(26), p. e2422633122. Available at: https://doi.org/10.1073/pnas.2422633122.

Collins Dictionary (2025) *Collins' Word of the Year 2025: AI meets authenticity as society shifts*. Available at: https://blog.collinsdictionary.com/language-lovers/collins-word-of-the-year-2025-ai-meets-authenticity-as-society-shifts/ (Accessed: October 12, 2026).

Corbett, A.T. and Anderson, J.R. (1994) "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Modeling and User-Adapted Interaction*, 4(4), pp. 253–278. Available at: https://doi.org/10.1007/BF01099821.

Dai, M. *et al.* (2021) "Knowledge Tracing: A Review of Available Technologies," *Journal of Educational Technology Development and Exchange*, 14(2), pp. 1–20.

LeetCode Coach (2026) *LeetCode Coach: AI mentor that tells you exactly what to solve next*. Available at: https://www.producthunt.com/products/leetcode-coach (Accessed: October 13, 2026).

Lopez, M. *et al.* (2008) "Relationships between reading, tracing and writing skills in introductory programming," in *Proceedings of the ACM Workshop on International Computing Education Research (ICER '08)*, pp. 101–111. Available at: https://doi.org/10.1145/1404520.1404531.

OpenStax (2019) *8.3 A Firm's External Macro Environment: PESTEL*. Available at: https://openstax.org/books/principles-management/pages/8-3-a-firms-external-macro-environment-pestel (Accessed: October 20, 2026).

Shen, J.H. and Tamkin, A. (2026) *How AI Impacts Skill Formation*. Available at: https://doi.org/10.48550/arXiv.2601.20245.

Stack Overflow (2025) *2025 Developer Survey: AI*. Available at: https://survey.stackoverflow.co/2025/ai (Accessed: October 12, 2026).

Tabassi, E. (2023) *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. technical report NIST AI 100–1. Available at: https://doi.org/10.6028/NIST.AI.100-1.